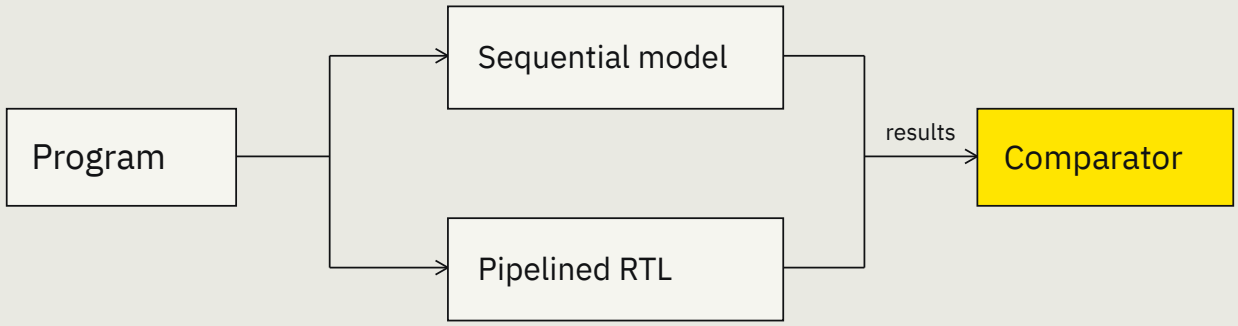
# VOLUME 06 / 08

# VERIFY THE CORE

## Build an oracle, design tests and interpret coverage

Run differential tests, verify memory writes, add directed cases, preserve seeds and measure a named coverage metric.

FIG 10 / TWO EXECUTIONS, ONE COMPARISON




Same program words drive both implementations. A mismatch identifies the first divergent result.

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

# What you will build

You will build evidence that each supported instruction produces the intended outcome under the stated memory contract. Distinguish a test pass from coverage and certification.

### Before you start

Understand the instruction contract and hazards. Know how to read a trace.

### Files and outputs

tests/run.py, tests/tb.sv, tests/coverage_main.cpp, scripts/coverage.py and reports/. Teaching labs write only build/learning-series/.

### Build order

1. Write architectural predictions independent of stage timing.
2. Drive program words into the RTL.
3. Compare ordered retirements and byte writes.
4. Add directed cases and seeded random programs.
5. Collect and explain core-only coverage.
6. Plan stronger external checks.

---

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

# A reference model predicts architectural results

A sequential model executes one instruction at a time without implementing the pipeline. For each instruction, it calculates the next PC, destination register, result and trap behavior.

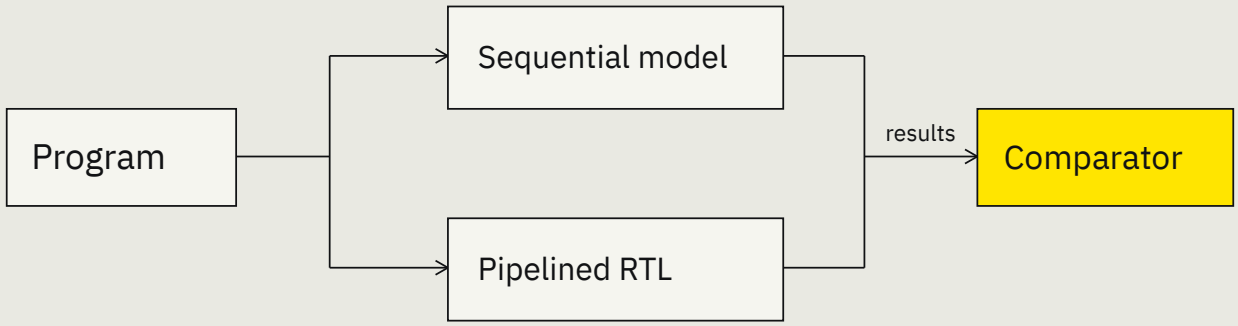
The regression runs the same machine words through that model and the RTL. It compares each retired PC, instruction, destination, value and trap flag. It separately compares every ordered byte write.

This catches errors that a final signature can hide. A wrong-path store might briefly corrupt memory and later be overwritten. A final register check alone can miss it.

The model uses Python integers and explicit 32-bit masking. Signed conversion is deliberate. Python's unlimited integer width does not automatically behave like a 32-bit register.

Both model and RTL belong to this project. Independent code does not guarantee an independent interpretation of the ISA. Hand-calculated examples, external models, ACT4 and formal properties provide additional evidence.

FIG 10 / TWO EXECUTIONS, ONE COMPARISON




Same program words drive both implementations. A mismatch identifies the first divergent result.

# Tests should target a reason the core could fail

A **directed test** constructs a known corner case. A **randomized test** explores many combinations using a reproducible seed. Both need an expected result.

For forwarding, cross source operand with producer distance and producer type. Test competing writers, x0 and a source field that is actually an immediate. For loads, cover each size, signedness and aligned byte position.

For control flow, check both outcomes of every branch, JAL, JALR, backward loops and wrong-path side effects. For traps, put a store after the fault and prove it never writes.

An **assertion** checks an invariant during execution. Useful future assertions include x0 stability, no writes from invalid stages, front-end stability on a stall and younger-stage squash on a redirect.

> **NEGATIVE EXPERIMENT**
>
> Predict a failing program, deliberately break one forwarding selection in a separate temporary worktree, and observe the first mismatch. Restore the good implementation. Preserve the failure explanation.

Mid-flight reset tests, formal integration and a complete hazard cross-coverage plan remain follow-up work. Do not describe them as already checked.

# A percentage needs a denominator

**Measured reference result:** Verilator 5.020 reports 37 of 38 core line instrumentation points hit, or 97.37%, without exclusions. Its uncovered point is the invalid branch-funct3 default.

The source is compact, so 38 points are a small denominator. Hitting a line does not test all operands or timing interactions. This metric is narrower than the phrase “97% correct.”

| Metric | Reference result | Scope |
|---|---|---|
| Passing programs | 144 / 144 | Local directed and random suite |
| Retired instructions | 7,358 checked | Ordered architectural trace |
| Line instrumentation | 37 / 38 = 97.37% | Core only, Verilator 5.020 |
| Functional bins | 46 / 46 | Named class/outcome plan |

The regression also hits 46 of 46 named functional bins. That plan covers instruction classes, branch outcomes and trap notification. It does not cover all hazard combinations. The 144 passing programs and 7,358 retired instructions describe the reference run, not every future version.

Adding tests changes counts. Changing source or tool versions can change the line denominator. Reproduce and report your own numbers after edits. Coverage is evidence of exercised behavior, not certification.

# Step 1: define the oracle state

The model needs 32 integer registers, byte-addressed data memory, PC and a stopping rule. It executes sequentially. It must not model forwarding or pipeline bubbles to predict architectural results.

```
regs = [0] * 32
memory = bytearray(16384)
pc = 0
# Per instruction:
# decode → compute result and next PC → apply permitted effects
# mask register results to 32 bits, then keep x0 equal to zero
```

Use explicit signed conversion for signed comparisons, immediate interpretation and byte lanes. Apply 32-bit masking at architectural boundaries. Derive operations from the written contract, not simulator output.

For each instruction, record PC, word, retained destination/value and trap. For stores, record each written byte and address. Distinguish ordinary completion from a local halt/trap notification.

Start with hand-calculated programs before random generation. Two implementations can agree on the same wrong interpretation.

# Step 2: drive the actual RTL

The supplied testbench loads a hex word file into instruction memory and supplies a byte-addressed data array. Each line contains one eight-digit hexadecimal instruction, without an address prefix.

```
iverilog -g2012 -s tb -o build/sim \
  rtl/rv32_core.sv tests/tb.sv
vvp build/sim +program=build/forwarding.hex
```

These native-tool commands run inside the tools container. Run `make test` first to create the program and executable, or create your own hex file explicitly.

The bench samples memory-write strobes on rising edges and prints retirement after updates settle. It stops on a trap record and has a bounded timeout. EBREAK is the ordinary test-end instruction in these programs.

Keep generated accesses within the memory range. The bench masks addresses; that convenience is not a hardware protection or access-fault feature.

# Step 3: compare the first divergence

Parse each RET record into a fixed tuple. Compare complete lists, including lengths. A design that produces a correct prefix then silently loses instructions must fail.

```
observed = [pc, insn, rd, value, trap]
expected = reference_instruction_record
assert observed == expected
# Also compare the complete ordered byte-write list.
```

This is explanatory pseudocode. The executable implementation is in `tests/run.py` and the teaching runner. A source-path mismatch or stale executable can otherwise make a test appear to check the wrong RTL revision.

When a test fails, identify the first differing PC and word. Decode it by hand, then inspect preceding producers and redirections. Store mismatches require a separate look at address, byte lane and forwarded data.

Save the failure input and seed before editing anything. Reduce it to the shortest reproducing program when possible.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 6A. Why can a final memory signature miss a wrong-path store that an ordered write-stream comparison catches?

# Step 4: add a named directed case

Use the supplied instruction encoders to construct a small, auditable program. This example is a teaching case, not an automatic addition to the baseline suite.

```
yield 'my_load_store', [
    imm(0, 1, 0, 9),
    store(2, 0, 1, 128),
    imm(2, 2, 0, 128, 3),
    store(2, 0, 2, 132),
    0x00100073,
]
```

Predict x2=9 and word 132=9. Explain why the load-to-store pair needs the interlock. Include x0, invalid encodings, backward control flow and both operand positions in later cases.

Add a case inside `suite()` on your learning branch. The normal runner's functional-bin assertion expects its complete suite, so a tiny standalone subset should use the separate learning runner instead of deleting all other cases.

# Step 5: use randomness reproducibly

A seed identifies a repeatable random sequence. Preserve the generator source revision along with the seed: changing generation code can change the program even with the same number.

Constrain addresses and legal encodings for ordinary random programs. Test illegal instructions separately, because the first expected trap ends execution and prevents later random instructions from exercising anything.

Use multiple register dependencies and mixed loads/stores. Keep a known reachable stopping instruction. A random jump generator needs control-flow constraints to avoid endless loops or out-of-range targets.

The reference suite uses twenty deterministic seeds plus directed cases. This is finite evidence, not exhaustive state-space exploration. Report program counts, retired records and the scope of the generator.

When a random failure appears, save its exact hex words. A failing program is more useful than a seed that no longer reproduces after a generator edit.

# Step 6: collect coverage with a declared scope

Run the existing coverage target with the tools container. It builds the Verilator testbench, runs the suite, merges per-program data and counts core line instrumentation points.

```
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD:/work" rv32-tools make coverage
```

Read `reports/line-coverage.json`: metric, hit, total, percent, scope, uncovered points, tool and RTL hash. Inspect the raw merged data if the parser finds no points after a tool update.

Add an invalid branch-funct3 case to exercise the current unhit default. Verify its trap and suppressed younger store, not just an increased percentage. A different source revision may have a different denominator.

Keep the 46-bin functional plan separate from RTL line coverage. Add meaningful hazard crosses deliberately and document the revised plan. Do not erase hard bins to improve a headline.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 6B. If a new simulator instruments more points, should you keep quoting the old 97.37%?

# Step 7: strengthen the evidence

Add invariants for x0, valid-controlled side effects, stable front-end state during a stall and younger squash on a redirect. Prove that a deliberate defect can trigger each assertion before relying on it.

Test reset while instructions are in flight. Decide how external memory treats reset and prevent unintended writes during that transition. Initial reset tests alone do not establish mid-flight reset correctness.

For external ISA checks, investigate ACT4 configuration, program loading and the DUT execution environment. For formal work, implement a complete RVFI adapter and state the exact properties and configurations checked.

The current project has neither integration. A partial retirement trace is not a completed formal interface, and local code coverage is not architectural certification.

Checkpoint: preserve one failure-to-fix example, a full passing regression, and a coverage report whose denominator you can explain.

# A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

| | |
|---|---|
| **Question** | What important behavior escapes the current coverage plan? |
| **Prediction** | A class/outcome bin can pass while a specific producer/consumer hazard remains untested. |
| **Experiment** | Add a dedicated missing hazard case and explain its independent expected result. |
| **Evidence** | Updated test, comparison results and unchanged or revised bin definitions. |

## What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

# Answers and completion check

**6A.** A later store can overwrite the wrong-path corruption before the final signature is checked. The ordered write list still records the forbidden event.

**6B.** No. Recompute and report the new numerator, denominator, version and source hash. Explain why the metric changed.

### Completion check

Reproduce a passing suite in both simulators, diagnose a deliberate mismatch, preserve seeds/words and explain the limits of every percentage. Never relabel finite simulation as proof or certification.

# Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. Verilator guide
2. Icarus documentation
3. Architectural tests
4. riscv-formal
5. Project source

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

## Keep the evidence boundary

Teaching labs are separate from the 144-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, external ISA certification, formal proof or matched application speedup is implied.

Continue with Volume 7. Run the complete regression before treating a learning edit as a finished implementation.